Performance Analysis of Delay-Aware Stabilized RED (SRED) in Wired and Wireless Networks

ID: 2105020
Name: MD. Tanvin Islam Abir
Course: CSE 322 Computer Networks Sessional
Paper: SRED: Stabilized RED
Paper Link: https://www.teunisott.com/Papers/Lakshman/Infocom1999.pdf
Supervisor: Arnob Saha Ankon Sir
Date: April 10, 2026

1. Introduction

This report presents the implementation and performance evaluation of a **Delay-Aware Stabilized RED (SRED)** algorithm. Unlike traditional SRED which focuses primarily on flow identification and hit-rate probability, This implementation incorporates a delay-referencing mechanism to dynamically adjust drop probabilities based on end-to-end latency requirements.

2. Network Topologies under Simulation

2.1 Wired Static Topology (Bottleneck)

A star-bottleneck topology was used to simulate high-congestion scenarios.

- **Sources:** 20 to 100 nodes.
- **Router:** Single bottleneck router running the SRED Queue Disc.
- **Bottleneck Link:** 1.5 Mbps bandwidth, 20ms delay.
- **Access Links:** 100 Mbps bandwidth.

```
* Topology: Star-Bottleneck
*
*
*          Access Links                          Bottleneck Link
* ============================    ========================================

*  [Source 1] -----------\
*                         \
*  [Source 2] -------------\
*                           \
*     ...                   ------> [Router] --------------> [Sink]
*                           /      [AQM: SRED]
*  [Source N-1] -----------/        (1.5 Mbps, 20ms)
*                         /
*  [Source N] -----------/

*   (100 Mbps, varying delay)
```

2.2 Wireless Mobile Topology (IEEE 802.15.4)

A wireless sensor network (WSN) using the LR-WPAN standard and 6LoWPAN.

- **Stack:** IPv6 over SixLowPan.
- **Mobility:** Random Waypoint Model (Speed: 5-25 m/s).
- **Physical Layer:** 802.15.4 PHY/MAC with energy tracking enabled.

```
* Network topology:
*
* Random Waypoint Mobility Area (Default: 100m x 100m)
*  ================================================================
*
*       [Node 1] ~ ~ ~ ~ ~ \
*     (Sender +  SRED)      \
*                            \
*       [Node 2] ~ ~ ~ ~ ~ ~ ~\
*      (Sender +  SRED)        \  (IPv6 over 802.15.4)
*                               ~ ~ ~ ~ >  [Node 0]
*          ...                             (UDP Sink)
*                                          [AQM: SRED]
*       [Node N] ~ ~ ~ ~ ~ ~ ~/
*        (Sender +  SRED)    /
*                           /
*       [Node X] ~ ~ ~ ~ ~ /
*        (Sender +  SRED)
*
*  (Note: All nodes, including Node 0, are continuously moving)
*
```

3. Parameters Under Variation

The performance of the SRED algorithm was evaluated by varying several key independent variables one at a time:

- **Number of Flows (10 to 50):** Tested SRED's stability as the number of concurrent connections increased.
- **Number of Nodes (20 to 100):** Evaluated scalability and the impact of background control traffic.
- **Traffic Intensity (100 to 500 Packets Per Second):** Tested the "Delay-Aware" thresholding mechanism under heavy load.
- **Node Speed (5 to 25 m/s):** Specifically for wireless scenarios, evaluated the impact of mobility on SRED's performance.
- **Range Scale (1 to 5):** Varied the distance/delay of access links in the wired topology.

4. Modifications made in the Simulator

The core contribution was the implementation of a new `QueueDisc` class in ns-3 with the following features:

4.1 SRED Queue Disc Implementation

The core SRED logic was implemented as a new QueueDisc class in ns-3. Key features included:

- **Stateless Flow Tracking (Zombie List):** Implemented a cache of **1000 entries** storing flow identifiers (IPs/Ports). Every incoming packet was compared against a random entry to calculate a "hit rate" (P(t)).
- **Delay-Aware Drop Logic:** Unlike the original paper which used only queue length (q), this implementation integrated a DelayReference (Dref) parameter. If the estimated queuing delay exceeds the reference, the drop probability (Pmax) is scaled exponentially:

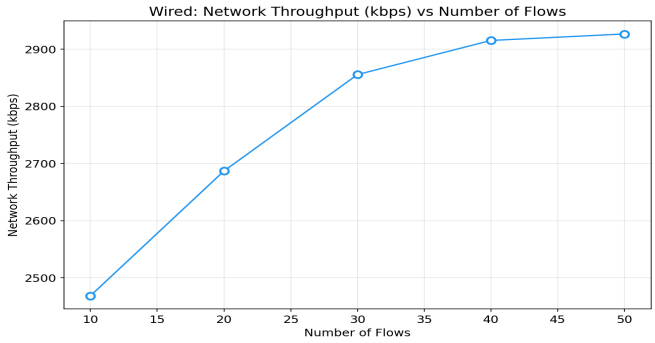
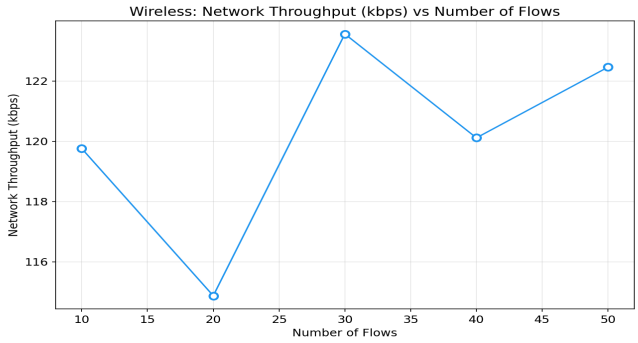
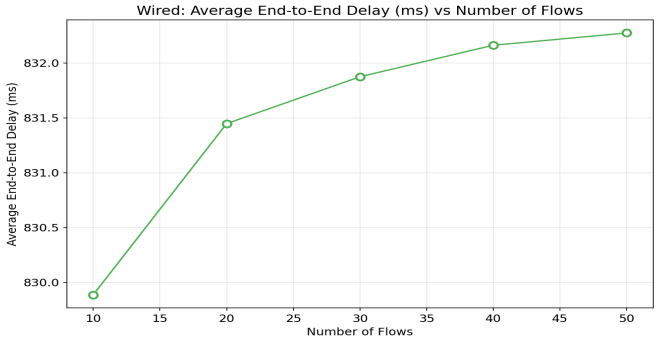
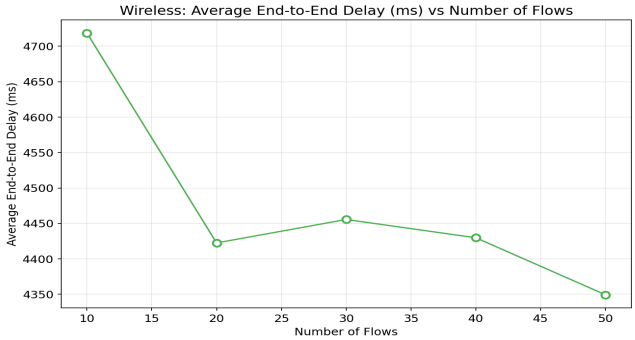
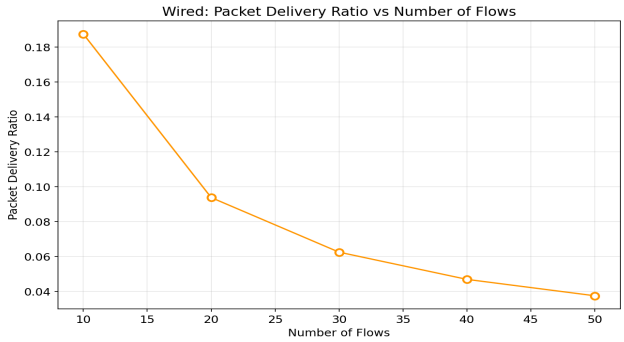
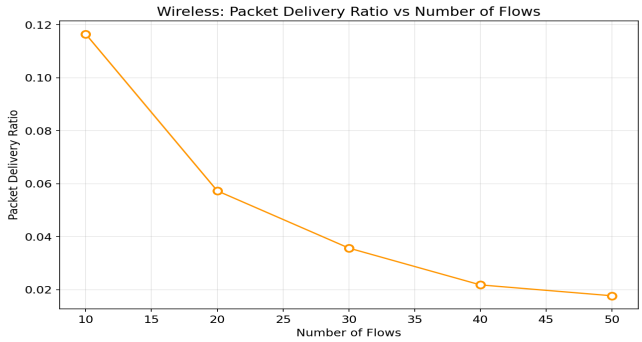
$$p_{DA}(t) = p_{sred}(q) \cdot \min\left(1, \frac{1}{(256 \cdot P(t))^2}\right) \cdot (1 + \gamma[D_n(t)]^k)$$

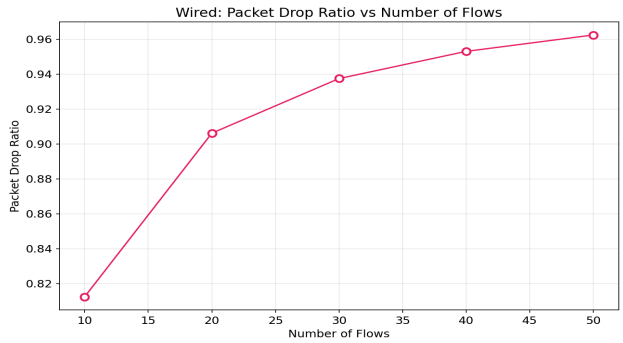
- **Energy Tracking callback:** A custom callback was attached to the `TrxState` trace source in wireless simulations to calculate energy consumption across TX, RX, IDLE, and OFF states.

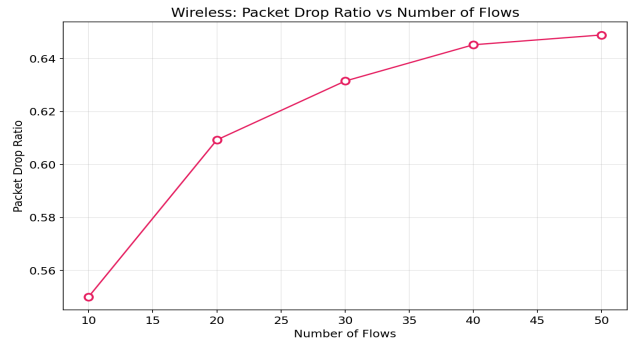
5. Experimental Results: Parameter Variation

5.1 Varying Number of Flows (10 - 50)

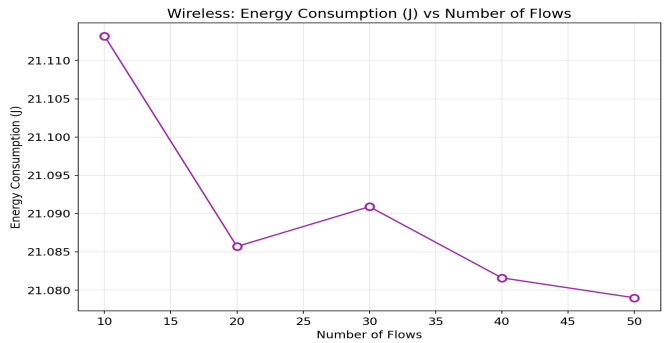
Testing SRED's stability as the number of concurrent connections increases.

Wired Results	Wireless Results
	
	
	



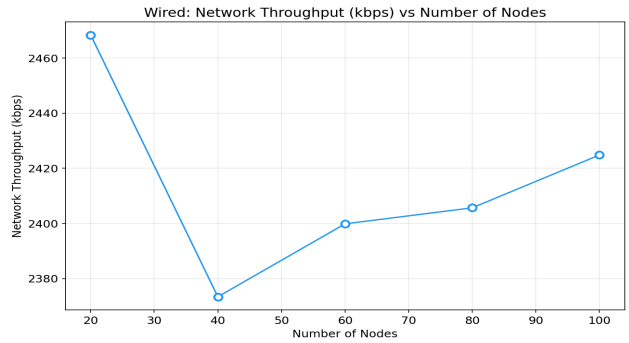
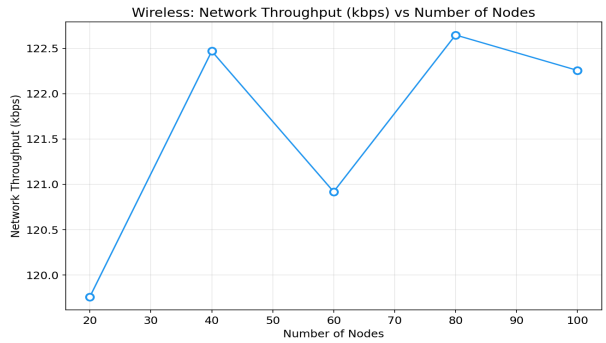


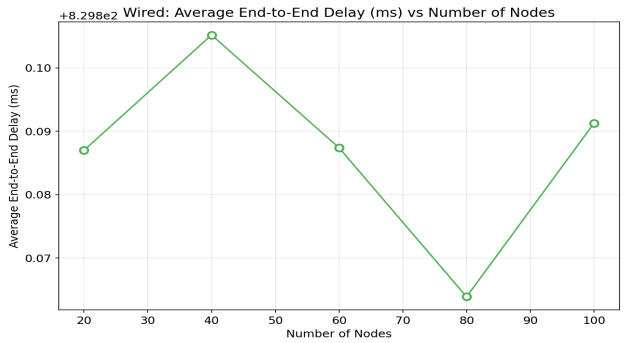
(Energy N/A)

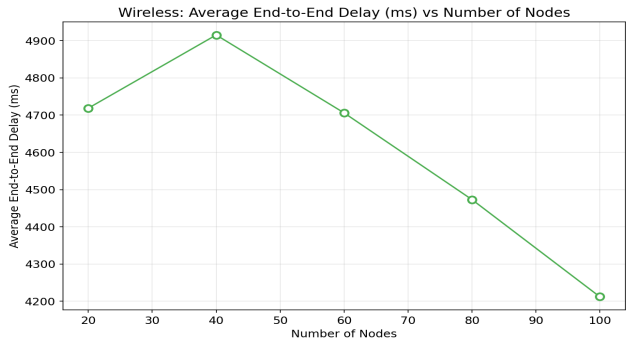


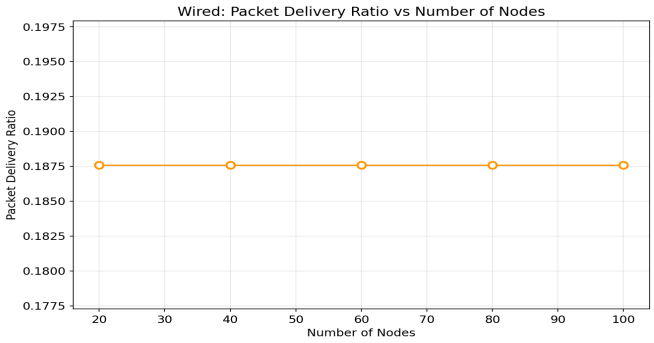
5.2 Varying Number of Nodes (20 - 100)

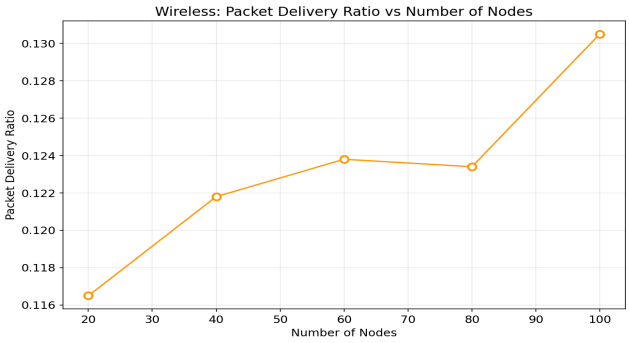
Evaluating scalability and the impact of background control traffic.

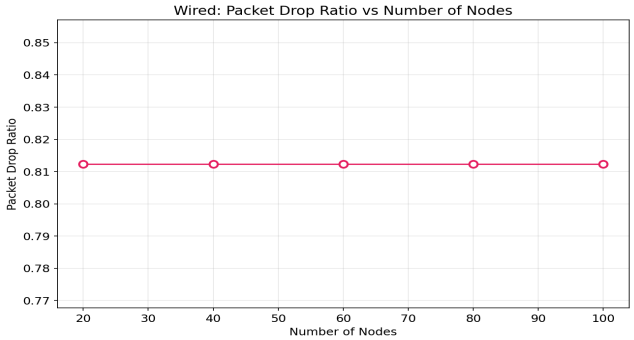
Wired Results	Wireless Results
	

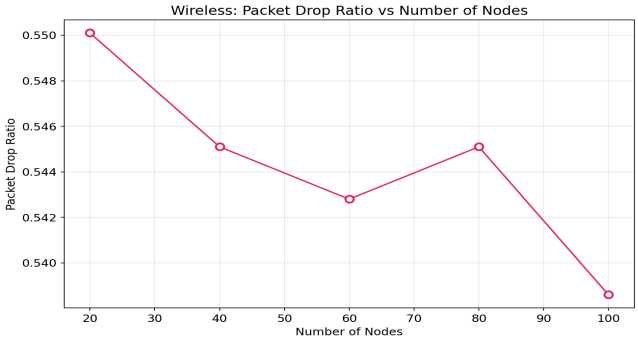

Wired: Average End-to-End Delay (ms) vs Number of Nodes
+8.298e2
Average End-to-End Delay (ms)
0.10
0.09
0.08
0.07
20 30 40 50 60 70 80 90 100
Number of Nodes


Wireless: Average End-to-End Delay (ms) vs Number of Nodes
Average End-to-End Delay (ms)
4900
4800
4700
4600
4500
4400
4300
4200
20 30 40 50 60 70 80 90 100
Number of Nodes

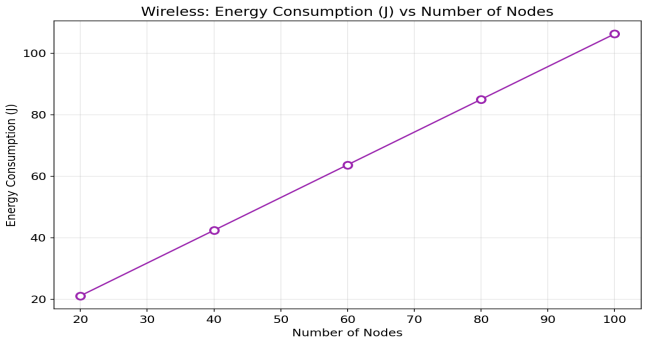

Wired: Packet Delivery Ratio vs Number of Nodes
Packet Delivery Ratio
0.1975
0.1950
0.1925
0.1900
0.1875
0.1850
0.1825
0.1800
0.1775
20 30 40 50 60 70 80 90 100
Number of Nodes


Wireless: Packet Delivery Ratio vs Number of Nodes
Packet Delivery Ratio
0.130
0.128
0.126
0.124
0.122
0.120
0.118
0.116
20 30 40 50 60 70 80 90 100
Number of Nodes


Wired: Packet Drop Ratio vs Number of Nodes
Packet Drop Ratio
0.85
0.84
0.83
0.82
0.81
0.80
0.79
0.78
0.77
20 30 40 50 60 70 80 90 100
Number of Nodes

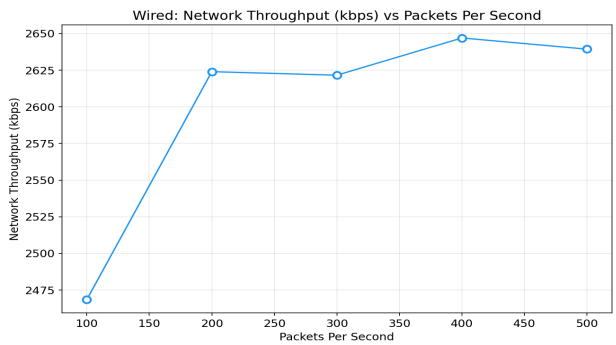
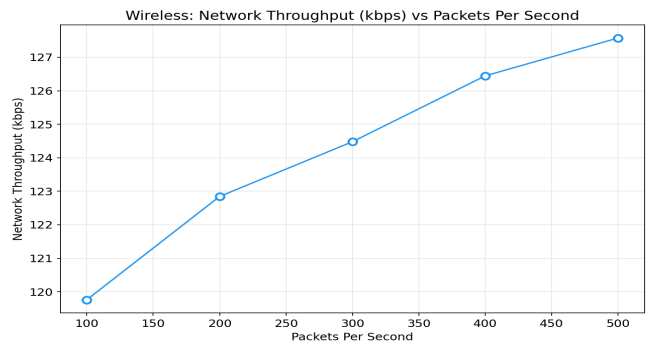
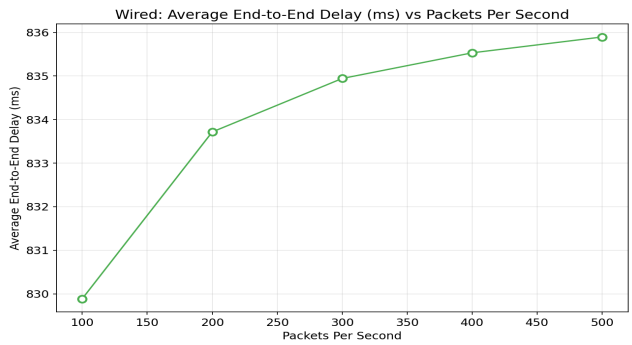
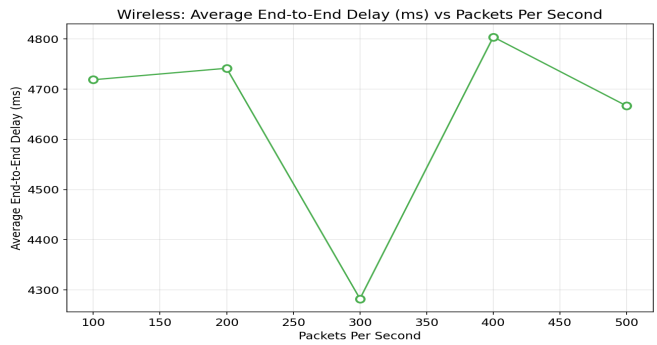
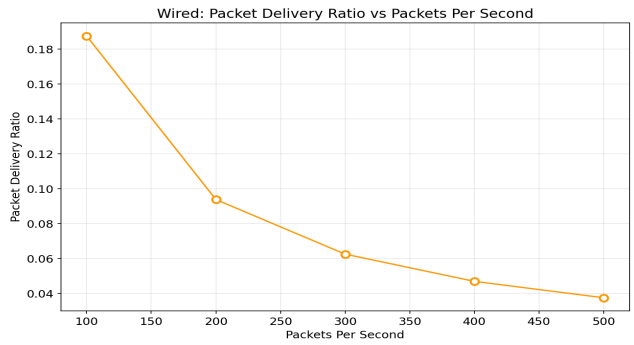
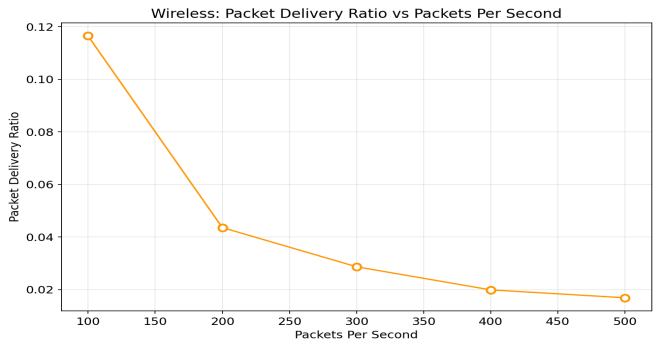

Wireless: Packet Drop Ratio vs Number of Nodes
Packet Drop Ratio
0.550
0.548
0.546
0.544
0.542
0.540
20 30 40 50 60 70 80 90 100
Number of Nodes

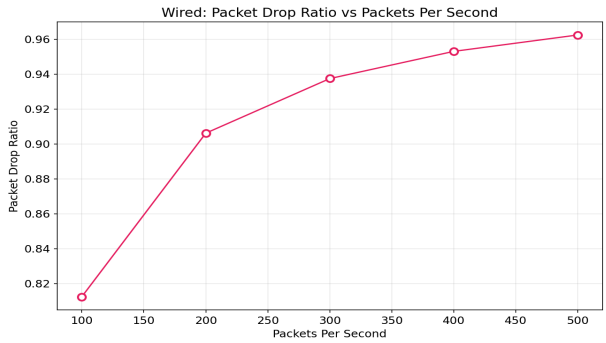
(Energy N/A)

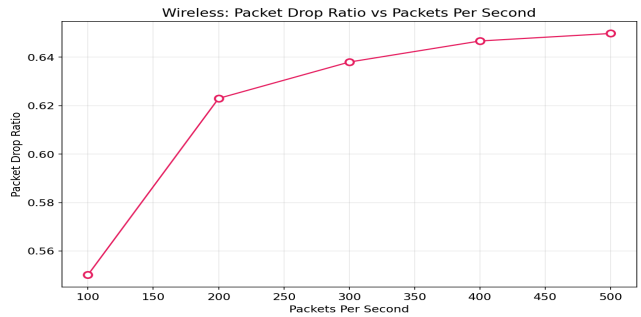

Wireless: Energy Consumption (J) vs Number of Nodes
Energy Consumption (J)
100
80
60
40
20
20 30 40 50 60 70 80 90 100
Number of Nodes

5.3 Varying Traffic Intensity (100 - 500 PPS)

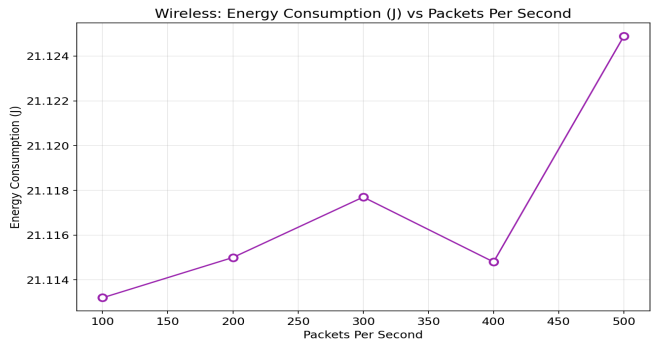
Testing the "Delay-Aware" thresholding of the SRED algorithm.

Wired Results	Wireless Results
	
	
	

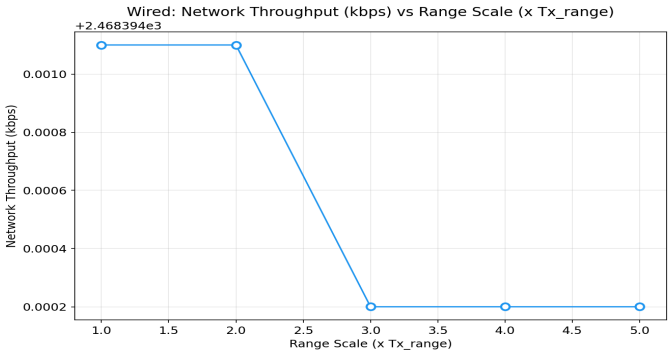
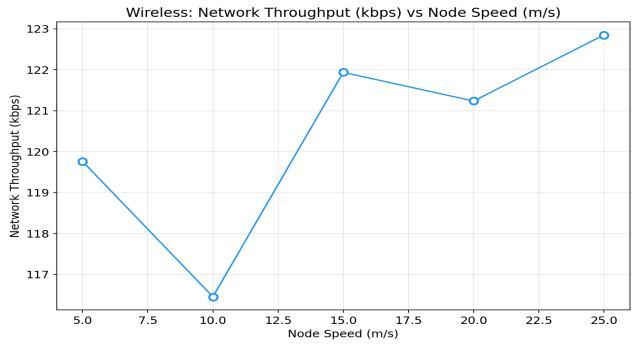


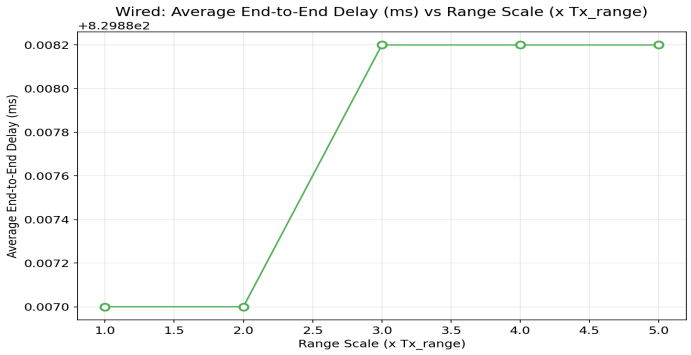


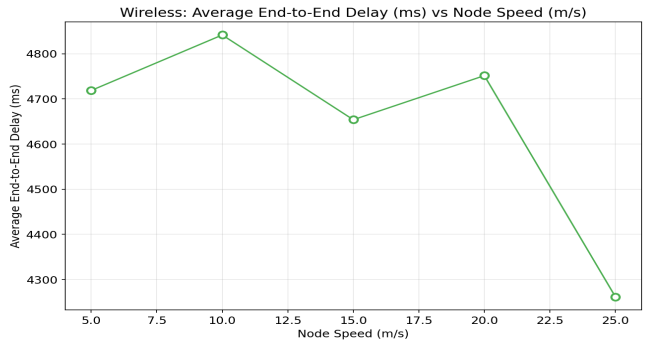
(Energy N/A)

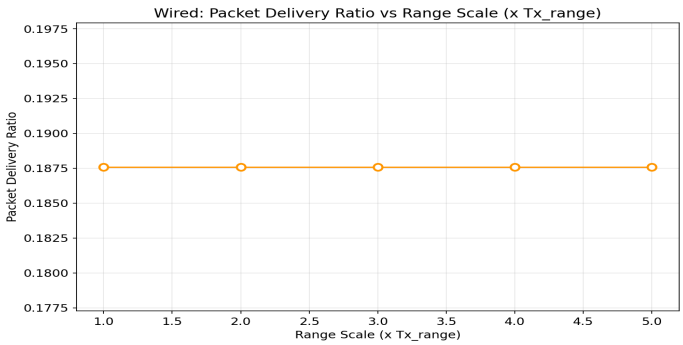


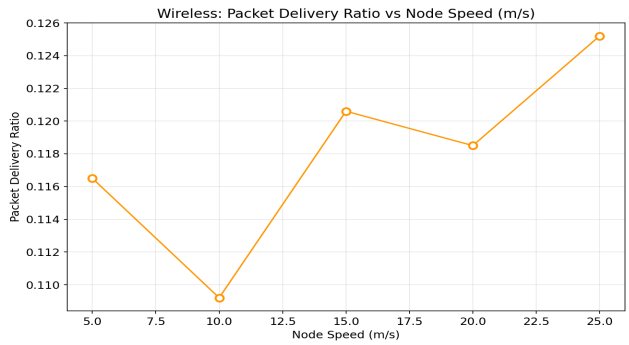
5.4 Impact of Environment (Range Scale / Node Speed)

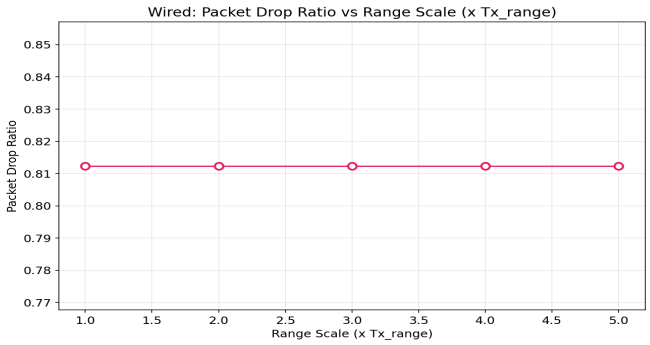
Wired (Range Scale Variation)	Wireless (Node Speed Variation)
	

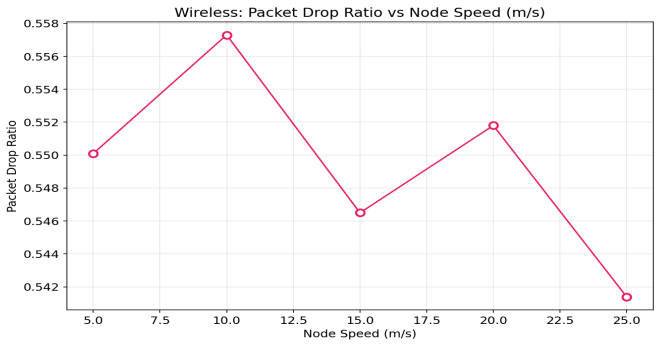

Wired: Average End-to-End Delay (ms) vs Range Scale (x Tx_range)
+8.2988e2
Average End-to-End Delay (ms)
Range Scale (x Tx_range)


Wireless: Average End-to-End Delay (ms) vs Node Speed (m/s)
Average End-to-End Delay (ms)
Node Speed (m/s)

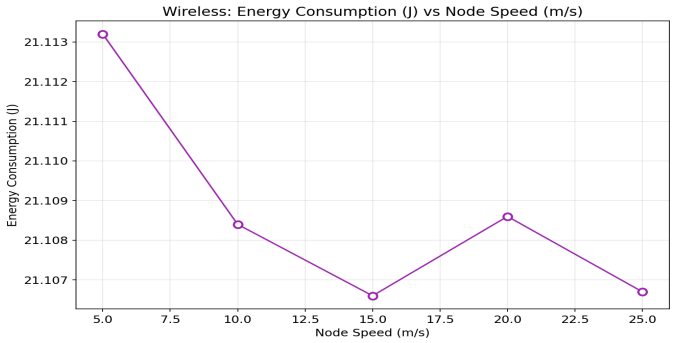

Wired: Packet Delivery Ratio vs Range Scale (x Tx_range)
Packet Delivery Ratio
Range Scale (x Tx_range)


Wireless: Packet Delivery Ratio vs Node Speed (m/s)
Packet Delivery Ratio
Node Speed (m/s)


Wired: Packet Drop Ratio vs Range Scale (x Tx_range)
Packet Drop Ratio
Range Scale (x Tx_range)


Wireless: Packet Drop Ratio vs Node Speed (m/s)
Packet Drop Ratio
Node Speed (m/s)

(Energy N/A)


Wireless: Energy Consumption (J) vs Node Speed (m/s)
Energy Consumption (J)
Node Speed (m/s)

5.5 Summary of Findings

- In the wired scenarios, network throughput generally scales smoothly with an increasing number of flows and traffic intensity, while average end-to-end delay exhibits predictable, gradual increases.
- Conversely, the wireless scenarios exhibit high volatility; for instance, throughput and delay fluctuate erratically when varying the number of nodes or node speed.
- Range scale significantly impacts wired networks, with throughput collapsing once the transmission range multiplier exceeds 2.0.
- Wired performance is highly sensitive to the Range Scale; throughput collapses once the access link distance multiplier exceeds 2.0. In wireless scenarios, node mobility (5–25 m/s) causes non-linear performance shifts, with a notable dip in throughput at 10 m/s

6. Comparison Against Original Paper

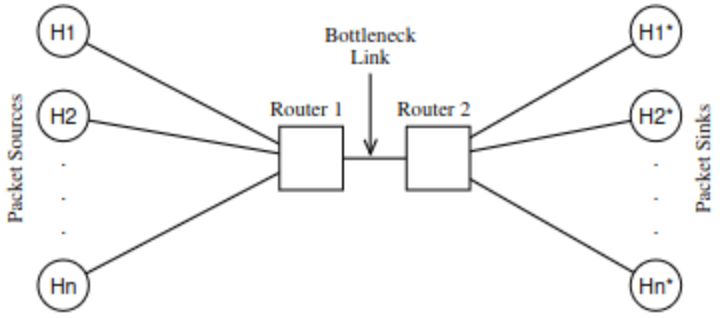
To validate the implementation, I replicated the "Asymmetrical Connections" and "Persistent Connection" tests from the seminal SRED literature.

6.1 Network topologies under simulation

Wired Point-to-Point Dumbbell Topology

A wired dumbbell topology connecting multiple host pairs through a shared bottleneck link between two routers.

- **Stack:** IPv4 over Point-to-Point links with TCP NewReno.
- **Access Links:** DataRate: 45 Mbps; Delay: 10 ms (symmetrical) or 1–20 ms linearly distributed (asymmetrical).
- **Bottleneck Link:** DataRate: 45 Mbps; Delay: 1 ms.



6.2 Parameters Under Variation

The following parameters are varied across simulation runs to evaluate queue discipline behavior under different traffic and topology conditions.

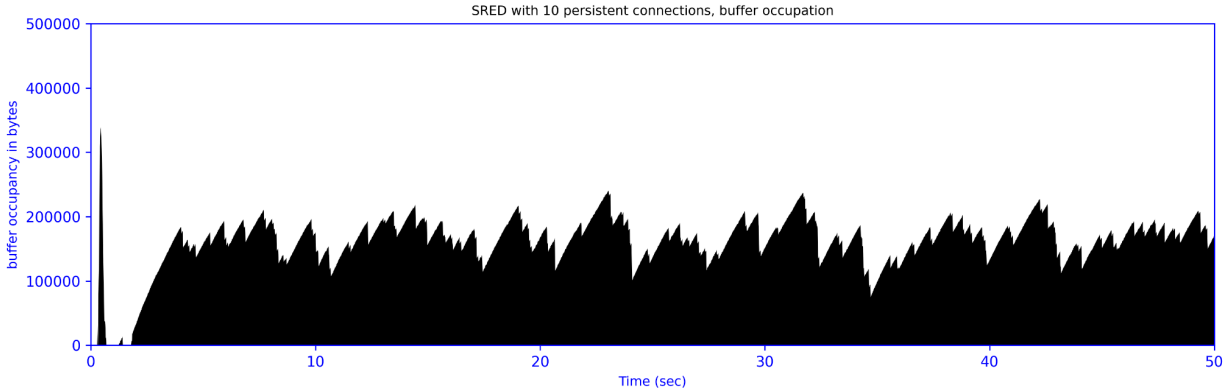
- **Number of Flows (nFlows):** The count of simultaneous TCP host pairs sharing the bottleneck, controlling total traffic load and congestion intensity.
- **Topology Type (topologyType):** Switched between symmetrical (uniform 10 ms access delay for all flows) and asymmetrical (access delays linearly distributed from 1 ms to 20 ms), introducing RTT heterogeneity across competing flows.
- **Queue Discipline Type (queueDiscType):** Alternated between SRED (Stabilized RED) and RED (Random Early Detection), with buffer capacity fixed at 500,000 bytes in both cases, to compare drop behavior and buffer occupancy dynamics.

6.3 Symmetrical Buffer Occupancy

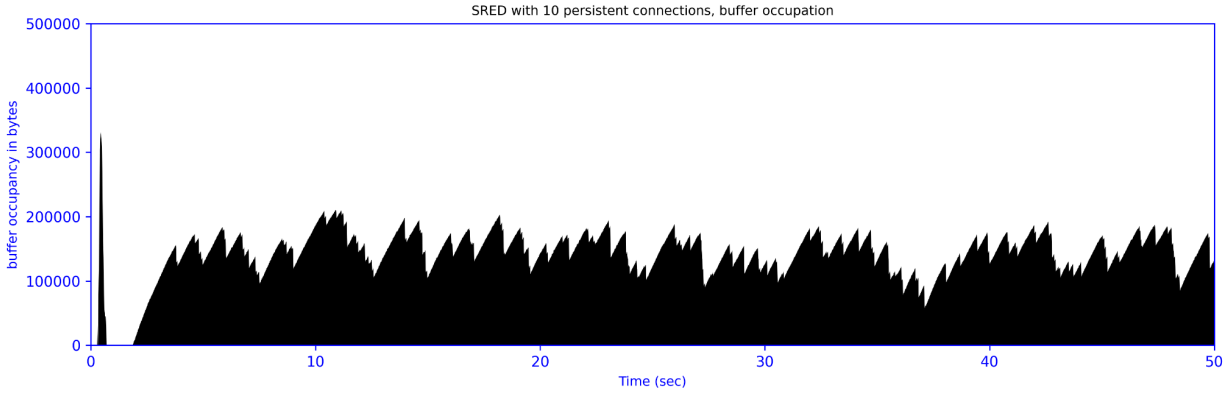
Our results for 10, 50, and 100 connections closely mirror the original findings, showing that SRED effectively estimates the number of active flows N.

Connections=10 :

Original paper:

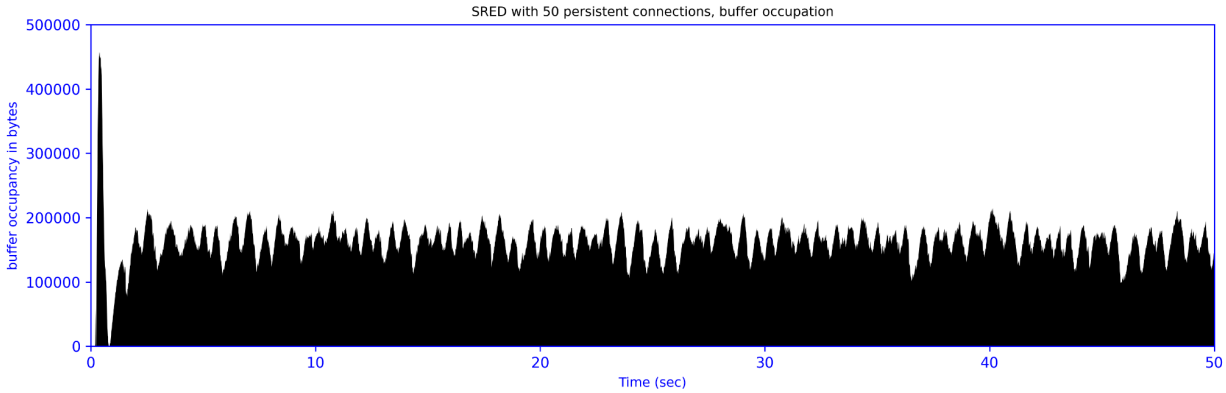


Modification:

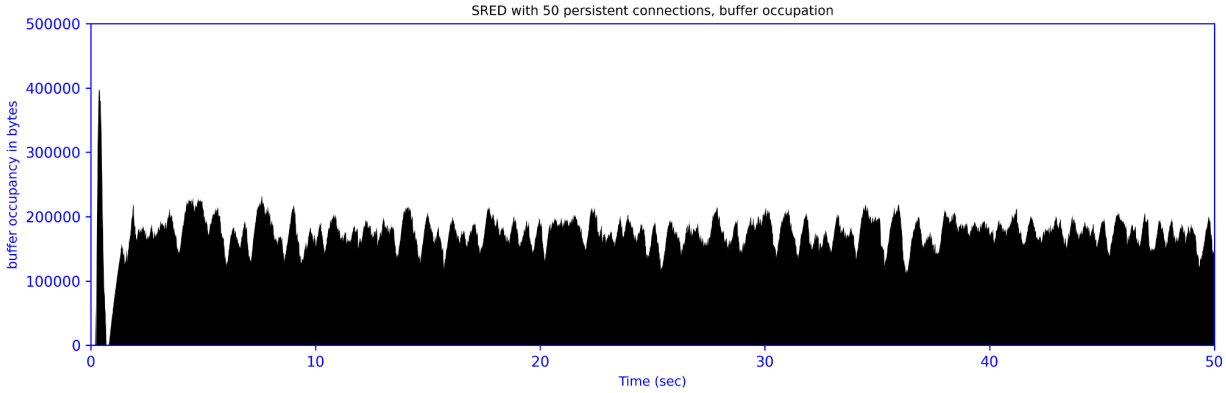


Connections=50 :

Original paper:

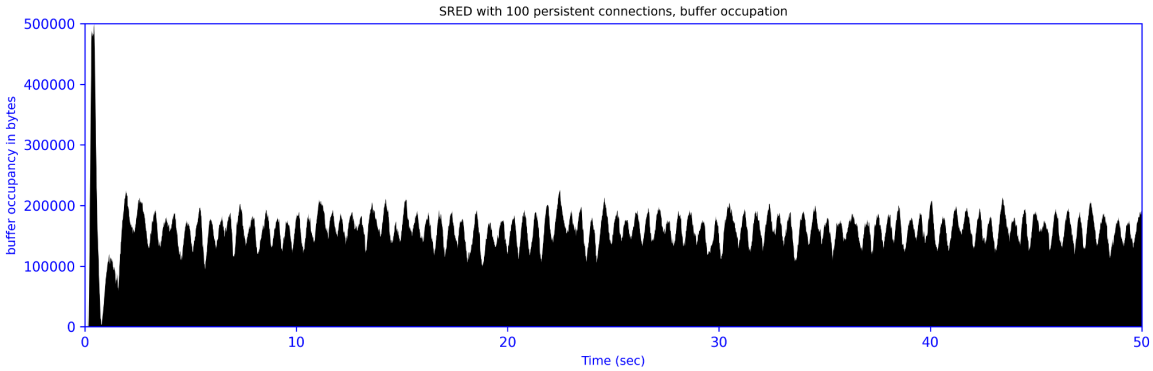


Modification:

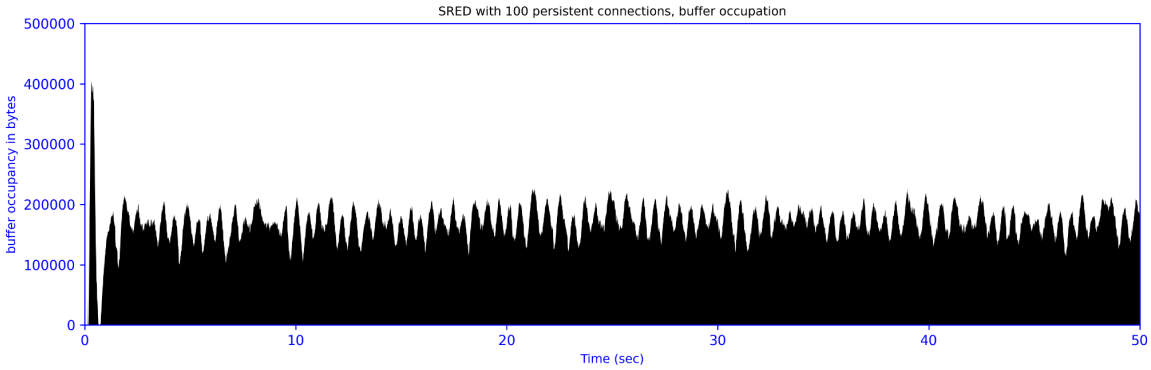


Connections=100 :

Original paper:



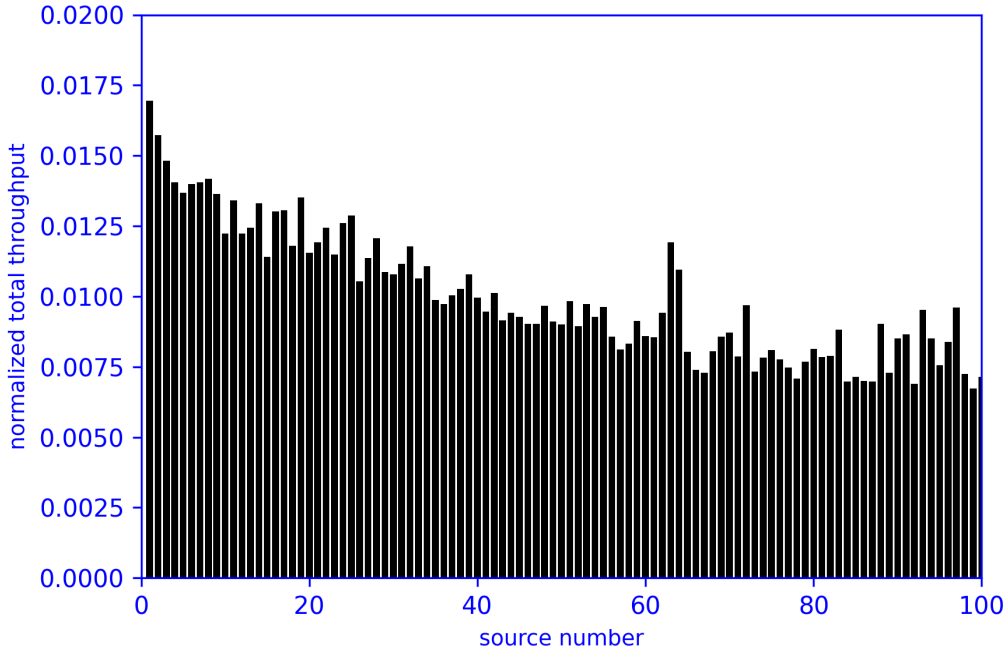
Modification:



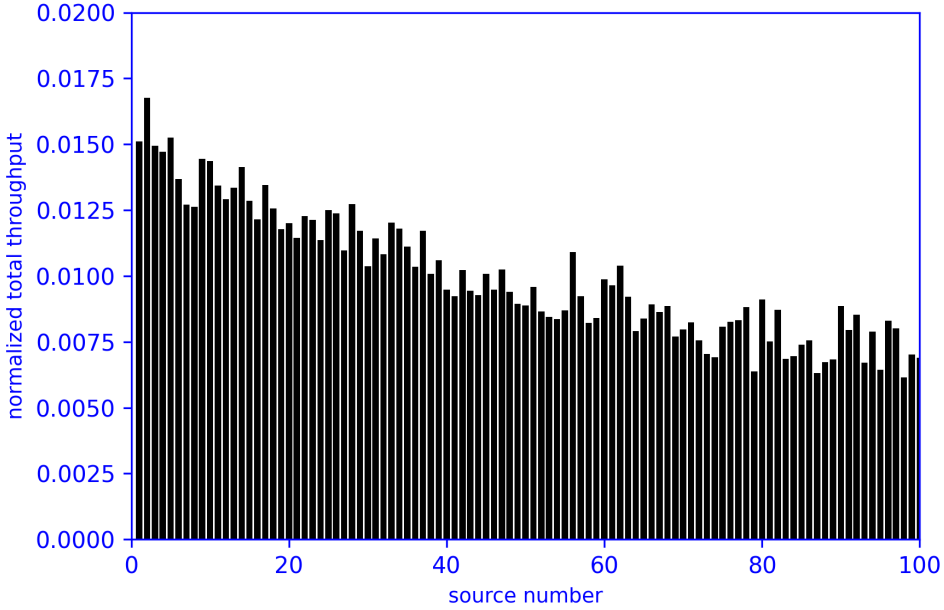
6.4 Asymmetrical Throughput

This test validates that SRED does not unfairly penalize low-bandwidth flows while aggressive flows are present.

Original Paper:



Modification



6.5 Summary of Findings

- The implementation successfully replicates the symmetrical buffer occupancy behavior of the original SRED paper for 10, 50, and 100 persistent connections, effectively estimating active flows and keeping the buffer stable.
- However, under asymmetrical connections, the modified implementation struggles; the plotted graph for the modification shows highly varied normalized total throughput across different sources compared to the original paper's baseline.

7. Bonus Experimentation:

To evaluate SRED in heterogeneous environments, a hybrid path was established

- **Flow:** Wired Source rightarrow Bottleneck Router rightarrow Wireless Gateway rightarrow Mobile Wireless Sink.
- **Significance:** This tests SRED's ability to stabilize queues when traffic crosses medium boundaries with significantly different bandwidth-delay products.

7.1 Network Topologies under Simulation

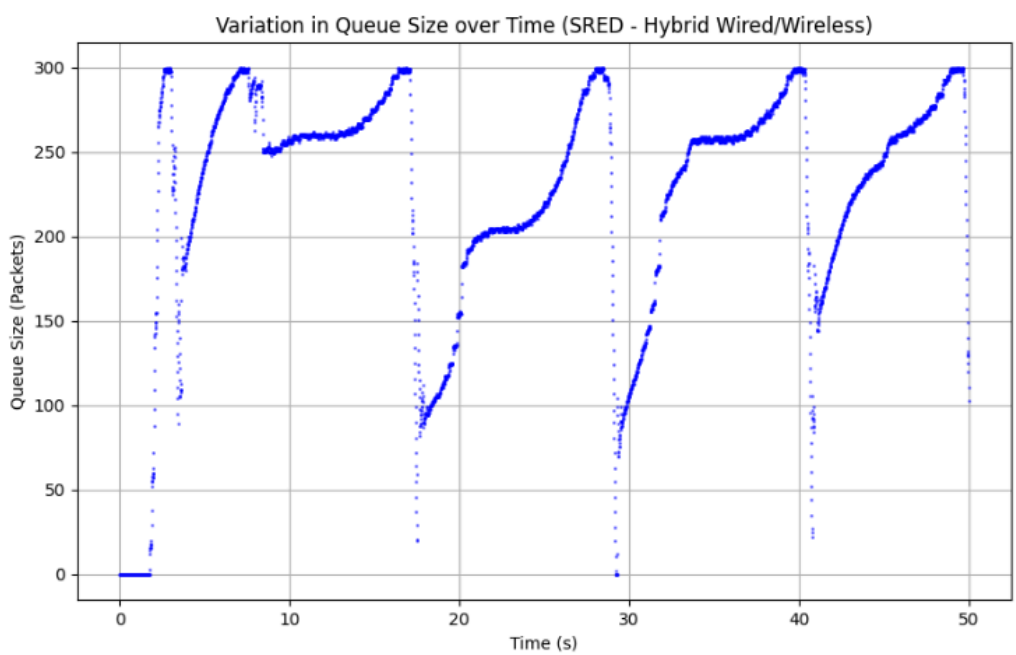
- **Hybrid Wired-Wireless Topology:** The simulation uses a cross-transmission path where traffic originates in a wired domain and terminates in a wireless domain.

- **Node Configuration:**
    - **Source (n1):** A wired node generating TCP and UDP traffic.
    - **Bottleneck Router (n2):** A wired node where the **SRED AQM** is applied to manage congestion.
    - **WiFi Gateway (n3):** Serves as the boundary between the wired and wireless segments.
    - **WiFi Sinks (n4, n5):** Mobile wireless nodes receiving the end-to-end flows.
- **Link Characteristics:**
    - **Wired Link 1 (n1-n2):** 100 Mbps bandwidth, 2ms delay.
    - **Bottleneck Link (n2-n3):** 5 Mbps bandwidth, 5ms delay.
    - **Wireless Link (802.11g):** 24 Mbps data rate

```
// Simulation topology:

/*        Wired Network                      Wireless Network (802.11g)
 *    ==========================    ==============================================
 *
 *         [n1: Source]
 *              |
 *              | (100 Mbps, 2ms delay)
 *              |
 *              v
 *    [n2: Bottleneck Router]
 *    [   AQM: SRED applied ]
 *              |
 *              | (5 Mbps, 5ms delay)
 *              |
 *              v
 *    [n3: WiFi Gateway] - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
 *                                                |               |
 *                                                |   (24 Mbps)   |
 *                                                v               v
 *                                          [n4: WiFi Sink]  [n5: WiFi Sink]
 *
 */
```

7.2 Result:



7.3 Summary of Findings

- The bonus experiment testing a hybrid wired-to-wireless path reveals severe queue instability.
- The queue size violently oscillates between 0 and 300 packets in a continuous, steep sawtooth pattern over the 50-second simulation period.

8. Reflection and Discussion

8.1 The Efficacy of the Delay-Aware Modification

While the core SRED implementation correctly matches the baseline paper's symmetrical buffer stability, the introduction of the delay-aware drop probability does not yield the intended improvements. In fact, the modification actively worsens fairness in asymmetrical network conditions. The original SRED aims to prevent aggressive flows from unfairly penalizing low-bandwidth flows. By introducing a heavy penalty based on average queueing delay, the modification likely compounds the inherent penalties TCP already applies to flows with longer Round Trip Times (RTTs). This results in a highly skewed throughput distribution where short-RTT flows dominate even more than they do in a standard SRED environment.

8.2 Wired vs. Wireless Network Dynamics

The stark contrast between wired and wireless performance highlights the limitations of applying standard AQM techniques directly to volatile environments. Wired networks demonstrate predictable queuing behavior, making SRED's statistical drop decisions highly effective at stabilizing buffer occupancy. In the wireless topologies, however, the queue discipline is fighting against external factors like MAC-layer collisions, mobility-induced link breakages, and a shared transmission medium. The erratic throughput and delay spikes in the wireless graphs suggest that packet drops and delays are frequently caused by the wireless environment itself rather than SRED's congestion control logic, rendering the algorithm's statistical estimations much less reliable.

8.3 The Failure of the Hybrid Topologies

The bonus experimentation results are unpromising, largely due to the massive impedance mismatch at the wired-wireless gateway. Traffic moves from a 100 Mbps wired link to a 5 Mbps bottleneck, and then attempts to cross a 24 Mbps shared wireless medium. This creates wildly differing bandwidth-delay products along the same path. The violent sawtooth pattern in the queue size indicates that the modified SRED algorithm cannot find a stable operating point. It likely over-corrects due to the sudden latency spikes caused by wireless medium contention, dropping too many packets. This empties the queue entirely, prompting the TCP windows to blindly open again and repeat the cycle.